DEVOPS OPERATIONS MANUAL

Version 1.0 • April 24, 2026

CI/CD Pipeline Setup Guide

Oracle Cloud Infrastructure • Docker • GitHub Actions

What you will learn in this guide:

✓ Provision an OCI Always Free AMD instance ✓ Install & configure Docker
✓ Set up Docker Compose ✓ Build a GitHub Actions CI/CD pipeline
✓ Push images to a container registry ✓ Auto-deploy on every code push

Oracle Cloud • VM.Standard.E2.1.Micro (1 GB RAM, AMD64)
GitHub Actions • Docker Hub / GHCR • SSH Deployment

Contents

About This Manual

This manual provides a **complete, beginner-friendly walkthrough** for deploying a Dockerised application to an Oracle Cloud Infrastructure (OCI) Always Free AMD instance using GitHub Actions as the CI/CD engine.

Who This Guide Is For

You do **not** need prior experience with OCI, Docker, or GitHub Actions. Every step is explained from scratch—from signing up for OCI through to watching your first automated deployment succeed.

Technology stack covered:

Component	Details
Cloud Provider	Oracle Cloud Infrastructure (Always Free Tier)
Instance Type	VM.Standard.E2.1.Micro — 1 GB RAM, x86_64/AMD64
OS	Ubuntu 22.04 LTS (recommended)
Containerisation	Docker Engine + Docker Compose
CI/CD	GitHub Actions (hosted runners)
Registry	GitHub Container Registry (GHCR) or Docker Hub
Deployment	SSH-based pull & restart strategy

Estimated Time

Allow **3–5 hours** for a first-time reader to complete all steps, including account sign-up waiting periods. Experienced users may complete it in 60–90 minutes.

Chapter 1

Architecture Overview

1.1 System Architecture

Before touching a terminal, it is valuable to understand how the components interact. Figure 1.1 shows the end-to-end pipeline.

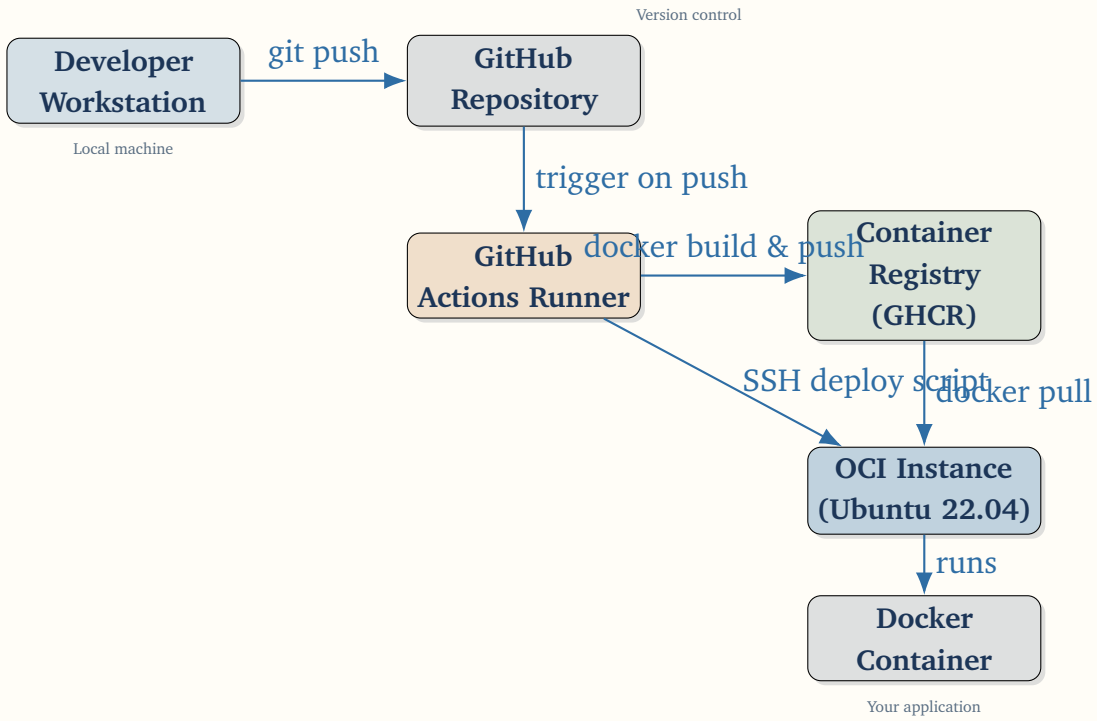


Figure 1.1: End-to-end CI/CD pipeline architecture

1.2 Pipeline Flow

1. A developer pushes code to the **main** (or any watched) branch on GitHub.
2. GitHub detects the push event and triggers an **Actions workflow**.
3. The Actions runner:
   (a) Checks out the code.

(b) Builds a **Docker image** using the project `Dockerfile`.

(c) Pushes the image to the **container registry**.

(d) Connects to the OCI instance via **SSH**.

(e) Runs a deploy script: pulls the new image and restarts the containers.

4. The OCI instance now serves the updated application.

1.3 Why This Approach?

Benefit	Explanation
Zero cost	OCI Always Free + GitHub's 2000 free Actions minutes/month
Reproducible	Docker guarantees identical environments
Automated	No manual SSH required after initial setup
Auditable	Every deployment is a GitHub Actions run with full logs
Scalable	Upgrade the OCI instance or add a registry with no pipeline changes

Chapter 2

Provisioning the OCI Instance

2.1 Creating an Oracle Cloud Account

Oracle Cloud offers a **permanently free** tier that includes two AMD-based compute instances. No credit card charges occur when you stay within Always Free limits.

1. Open your browser and navigate to https://cloud.oracle.com.
2. Click **Start for free** and complete the registration:
   - Enter your email address and verify it.
   - Provide your home country and home region (choose a region *close to your users*; you **cannot change** the home region later).
   - Enter payment details (for identity verification only; you will not be charged for Always Free resources).
3. Wait for the confirmation email (usually 5–15 minutes).
4. Sign in to the **OCI Console** at https://cloud.oracle.com.

Home Region is Permanent

Choose your home region carefully. Always Free resources must be created in your home region. Once set, it **cannot be changed**.

2.2 Generating an SSH Key Pair (Local Machine)

You will access the OCI instance via SSH. Generate a dedicated key pair now.

Listing 2.1: Generate SSH key pair (run on your local machine)

```
# Create the key pair (press Enter to accept default path)
ssh-keygen -t rsa -b 4096 -C "oci-instance-key"

# The default paths are:
#   Private key: ~/.ssh/id_rsa
#   Public key:  ~/.ssh/id_rsa.pub
```

```
7
8 # Display your PUBLIC key (you will paste this into OCI)
9 cat ~/.ssh/id_rsa.pub
```

Keep Your Private Key Safe

Never share `~/.ssh/id_rsa` (the private key). The public key (`id_rsa.pub`) is what you upload to OCI.

2.3 Creating the Compute Instance

1. In the OCI Console, open the navigation menu (top-left hamburger icon).
2. Navigate to **Compute** → **Instances**.
3. Click **Create instance**.
4. Fill in the form as shown in Table 2.1:

Table 2.1: Recommended instance creation settings

Field	Value
Name	`cicd-server` (or any name)
Compartment	Root compartment (default)
Image	Canonical Ubuntu 22.04 LTS
Shape	VM.Standard.E2.1.Micro (Always Free)
OCPU / RAM	1 OCPU / 1 GB RAM
Networking	Create a new VCN or use existing; enable public IP
SSH key	Paste contents of `~/.ssh/id_rsa.pub`

5. Click **Create**. The instance enters the *Provisioning* state.
6. Wait 2 minutes until the status shows **Running**.
7. Note the **Public IP address** displayed on the instance details page.

2.4 Opening Required Firewall Ports

OCI uses two layers of firewalling: the *VCN Security List* (cloud-level) and the *OS iptables/firewalld* (instance-level).

2.4.1 OCI Security List

1. In the instance details, click the **Subnet** link.
2. Click **Security Lists** → your default security list.
3. Add **Ingress Rules** for the ports your application needs:

Source CIDR	Protocol	Port	Purpose
0.0.0.0/0	TCP	22	SSH access
0.0.0.0/0	TCP	80	HTTP
0.0.0.0/0	TCP	443	HTTPS
0.0.0.0/0	TCP	8080	Custom app port (optional)

2.4.2 OS-Level Firewall

After connecting via SSH (next chapter), run:

Listing 2.2: Open ports in the Ubuntu firewall

```
sudo iptables -I INPUT -p tcp --dport 80  -j ACCEPT
sudo iptables -I INPUT -p tcp --dport 443 -j ACCEPT
sudo iptables -I INPUT -p tcp --dport 8080 -j ACCEPT

# Make rules persistent
sudo apt-get install -y iptables-persistent
sudo netfilter-persistent save
```

Chapter 3

Connecting to Your OCI Instance

3.1 First SSH Connection

Listing 3.1: SSH into the OCI instance

```
# Replace <PUBLIC_IP> with your instance's public IP address
ssh -i ~/.ssh/id_rsa ubuntu@<PUBLIC_IP>

# On first connection, type "yes" to accept the host fingerprint
# The authenticity of host '...' can't be established.
# RSA key fingerprint is SHA256:...
# Are you sure you want to continue connecting (yes/no)? yes
```

SSH Config Shortcut

Add this block to `~/.ssh/config` on your local machine to use `ssh oci` instead of the full command:

```
Host oci
  HostName <PUBLIC_IP>
  User ubuntu
  IdentityFile ~/.ssh/id_rsa
```

3.2 System Update

Always update the system before installing software:

Listing 3.2: Update system packages

```
sudo apt-get update && sudo apt-get upgrade -y
sudo apt-get install -y \
    curl wget git unzip ca-certificates \
    gnupg lsb-release apt-transport-https \
    software-properties-common
```

Chapter 4

Installing Docker on Ubuntu 22.04

4.1 Why Docker?

Docker packages your application and all its dependencies into a **container**—a lightweight, portable unit that runs identically everywhere. Figure 4.1 illustrates the layered architecture.

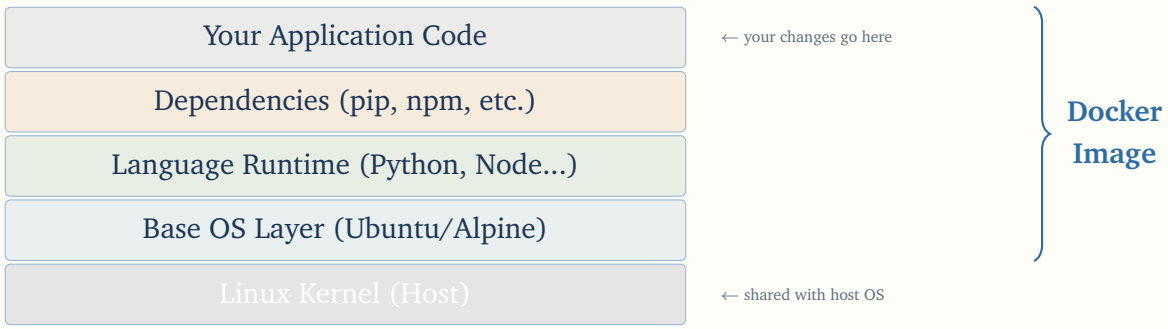


Figure 4.1: Docker image layered architecture

4.2 Remove Old Docker Versions

Listing 4.1: Remove conflicting Docker packages

```
sudo apt-get remove -y docker docker-engine docker.io containerd runc
    2>/dev/null || true
```

4.3 Install Docker Engine

Follow the official installation method using Docker's apt repository:

Listing 4.2: Add Docker's official GPG key and repository

```
# 1. Add Docker's official GPG key
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
  sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg
```

```
6
7  # 2. Add the Docker repository
8  echo \
9    "deb [arch=$(dpkg --print-architecture) \
10   signed-by=/etc/apt/keyrings/docker.gpg] \
11   https://download.docker.com/linux/ubuntu \
12   $(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \
13   sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
14
15 # 3. Update apt and install Docker Engine
16 sudo apt-get update
17 sudo apt-get install -y \
18     docker-ce \
19     docker-ce-cli \
20     containerd.io \
21     docker-buildx-plugin \
22     docker-compose-plugin
```

4.4 Post-Installation Configuration

Listing 4.3: Enable Docker and configure user permissions

```
1  # Start and enable Docker service
2  sudo systemctl enable --now docker
3
4  # Add your user to the docker group (avoids typing sudo each time)
5  sudo usermod -aG docker $USER
6
7  # Apply the group change in the current shell
8  newgrp docker
9
10 # Verify installation
11 docker --version
12 docker run hello-world
```

Log Out Required

The `usermod` change only takes full effect after you **log out and log back in**. The `newgrp docker` command applies it temporarily for the current session.

4.5 Verify Docker is Running

Listing 4.4: Check Docker service status

```
1  sudo systemctl status docker
2  # Look for:  Active: active (running) since ...
3
```

```
docker info
# Should display: Server Version: 24.x.x or later
```

Chapter 5

Installing Docker Compose

5.1 About Docker Compose

Docker Compose lets you define and run **multi-container** applications using a single YAML file. A typical web app has at least two containers: the application itself and a database. Compose manages them together.

Listing 5.1: Check if Docker Compose plugin is installed

```
# Docker Compose is now bundled as a plugin:
docker compose version
# Expected: Docker Compose version v2.x.x

# If the command fails, install the standalone binary:
COMPOSE_VERSION=$(curl -s \
  https://api.github.com/repos/docker/compose/releases/latest \
  | grep '"tag_name"' | cut -d'"' -f4)

sudo curl -L \
  "https://github.com/docker/compose/releases/download/${COMPOSE_VERSION}/docker-
     -s)-$(uname -m)" \
  -o /usr/local/bin/docker-compose

sudo chmod +x /usr/local/bin/docker-compose
docker-compose --version
```

Plugin vs Standalone

Modern Docker installations include Compose as a *plugin* (`docker compose`). Older systems use the standalone binary (`docker-compose` with a hyphen). Both work; this guide uses the plugin syntax.

Chapter 6

Structuring Your Application for Docker

6.1 Project Directory Layout

Organise your project as shown below. This layout is assumed throughout the rest of this guide.

Listing 6.1: Recommended project structure

```
my-app/
|-- .github/
|   \-- workflows/
|       \-- deploy.yml          # GitHub Actions workflow (Chapter 9)
|-- app/
|   |-- main.py                 # Your application code
|   \-- requirements.txt        # Python dependencies (example)
|-- nginx/
|   \-- nginx.conf              # Reverse proxy config (optional)
|-- Dockerfile                  # Container image definition
|-- docker-compose.yml          # Multi-container orchestration
|-- .dockerignore               # Files excluded from Docker build
\-- .env.example                # Example environment variables
```

6.2 Writing a Dockerfile

The `Dockerfile` tells Docker how to build your application image.

Listing 6.2: Example Dockerfile for a Python/Flask application

```
# -- Stage 1: Build dependencies ---------------------------------
FROM python:3.11-slim AS builder
WORKDIR /build

# Install build dependencies
COPY app/requirements.txt .
RUN pip install --no-cache-dir --prefix=/install -r requirements.txt

# -- Stage 2: Production image -----------------------------------
FROM python:3.11-slim
```

```
WORKDIR /app

# Copy installed packages from builder stage
COPY --from=builder /install /usr/local

# Copy application source
COPY app/ .

# Create a non-root user for security
RUN adduser --disabled-password --gecos "" appuser
USER appuser

# Expose application port
EXPOSE 8000

# Health check
HEALTHCHECK --interval=30s --timeout=10s --start-period=5s --retries=3 \
  CMD curl -f http://localhost:8000/health || exit 1

# Start command
CMD ["python", "-m", "gunicorn", "--bind", "0.0.0.0:8000", "main:app"]
```

6.3 Writing a docker-compose.yml

Listing 6.3: docker-compose.yml for a web app + database

```
version: "3.9"

services:
  app:
    image: ghcr.io/${GITHUB_USERNAME}/my-app:latest
    container_name: my_app
    restart: unless-stopped
    ports:
      - "8000:8000"
    environment:
      - DATABASE_URL=${DATABASE_URL}
      - SECRET_KEY=${SECRET_KEY}
    depends_on:
      db:
        condition: service_healthy
    networks:
      - app_network

  db:
    image: postgres:15-alpine
    container_name: my_db
    restart: unless-stopped
```

```
    environment:
      - POSTGRES_DB=${POSTGRES_DB}
      - POSTGRES_USER=${POSTGRES_USER}
      - POSTGRES_PASSWORD=${POSTGRES_PASSWORD}
    volumes:
      - db_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER}"]
      interval: 10s
      timeout: 5s
      retries: 5
    networks:
      - app_network

  nginx:
    image: nginx:alpine
    container_name: my_nginx
    restart: unless-stopped
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - ./nginx/nginx.conf:/etc/nginx/nginx.conf:ro
    depends_on:
      - app
    networks:
      - app_network

volumes:
  db_data:

networks:
  app_network:
    driver: bridge
```

6.4 The .dockerignore File

Exclude unnecessary files from the build context to speed up builds:

Listing 6.4: .dockerignore

```
.git
.github
__pycache__
*.pyc
*.pyo
.env
.env.*
!.env.example
node_modules
```

```
*.log
.DS_Store
README.md
```

Chapter 7

Setting Up the Container Registry

7.1 Choosing a Registry

A **container registry** is a storage service for Docker images. We recommend **GitHub Container Registry (GHCR)** because it integrates natively with GitHub Actions.

Registry	Free Tier	Best For
GHCR (GitHub)	Unlimited public; 500 MB private	GitHub-based projects
Docker Hub	1 private repo, unlimited public	General use
OCI Registry	500 MB Always Free	OCI-only projects

7.2 Using GitHub Container Registry (GHCR)

GHCR requires a **Personal Access Token (PAT)** for authentication.

1. Go to `https://github.com/settings/tokens?type=beta` (fine-grained tokens) **or** `https://github.com/settings/tokens` (classic).
2. Click **Generate new token (classic)**.
3. Select the following scopes:
   - `write:packages` — push images
   - `read:packages` — pull images
   - `delete:packages` — clean up old tags (optional)
4. Set an expiration (e.g., 90 days or No expiration for automation).
5. Click **Generate token** and **copy it immediately** (shown only once).

Store the Token Safely

Treat the PAT like a password. You will store it as a **GitHub Actions secret** in the next chapter. Never commit it to your repository.

Chapter 8

GitHub Repository and Secrets Setup

8.1 Creating the Repository

1. Go to https://github.com/new.
2. Name your repository (e.g., my-app).
3. Choose **Private** or **Public**.
4. Click **Create repository**.

Listing 8.1: Initialise Git and push to GitHub

```
cd my-app

# Initialise git (if not already done)
git init
git add .
git commit -m "Initial commit"

# Add remote and push
git remote add origin git@github.com:<YOUR_USERNAME>/my-app.git
git branch -M main
git push -u origin main
```

8.2 Configuring GitHub Actions Secrets

Secrets are encrypted variables available to your Actions workflow. They are never visible in logs.

8.2.1 Required Secrets

Navigate to your repository on GitHub: **Settings** → **Secrets and variables** → **Actions** → **New repository secret**.

Add each secret from Table 8.1:

Table 8.1: Required GitHub Actions secrets

Secret Name	Value
OCI_HOST	Public IP of your OCI instance
OCI_USER	ubuntu
OCI_SSH_KEY	**Full contents** of ~/.ssh/id_rsa (private key)
GHCR_PAT	GitHub Personal Access Token (with write:packages)
GHCR_USERNAME	Your GitHub username
DB_PASSWORD	Your database password
SECRET_KEY	Application secret key

Adding the SSH Private Key

Open the private key file and copy *everything* including the header and footer lines:

```
cat ~/.ssh/id_rsa
# Copy ALL output including:
# -----BEGIN RSA PRIVATE KEY-----
# ...
# -----END RSA PRIVATE KEY-----
```

Paste this complete content as the value of the OCI_SSH_KEY secret.

8.3 Setting Up the OCI Instance for SSH Deployment

GitHub Actions will SSH into your OCI instance using the key pair created earlier. Verify the public key is present:

Listing 8.2: Verify authorised keys on OCI instance (run on OCI)

```
cat ~/.ssh/authorized_keys
# Your public key should appear here.
# OCI adds it automatically during instance creation.
```

Listing 8.3: Create a deployment directory (run on OCI)

```
# Create a directory for your application
mkdir -p ~/apps/my-app
cd ~/apps/my-app

# Create a production .env file
nano .env
# Add your environment variables (DATABASE_URL, SECRET_KEY, etc.)
```

Chapter 9

Writing the GitHub Actions Workflow

9.1 Workflow File Location

GitHub Actions workflows are YAML files stored in `.github/workflows/` in your repository. Create the directory and the workflow file:

Listing 9.1: Create the workflow directory

```
mkdir -p .github/workflows
touch .github/workflows/deploy.yml
```

9.2 Complete CI/CD Workflow

The following workflow builds and deploys your application automatically on every push to the `main` branch.

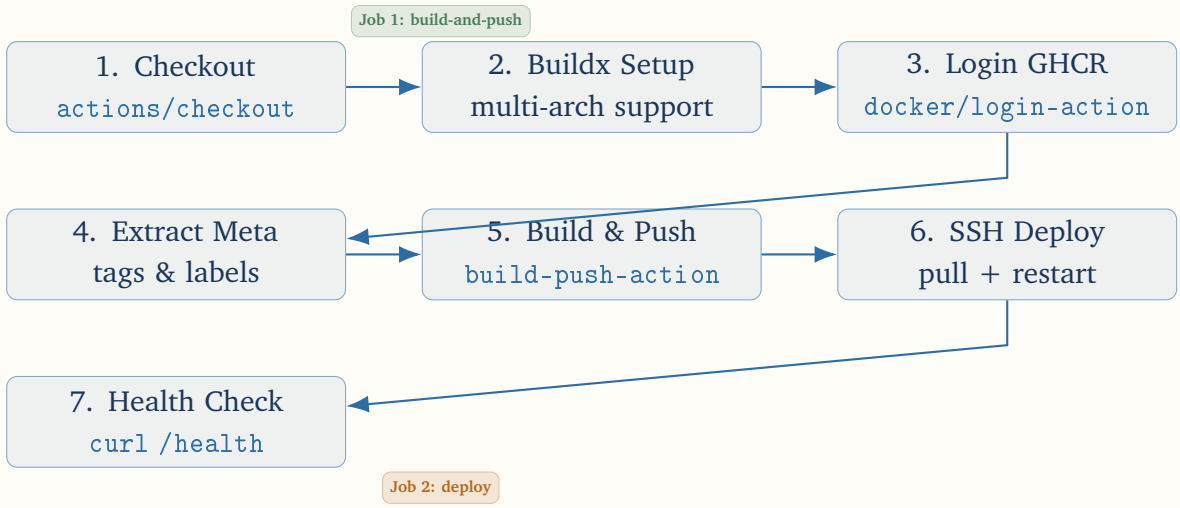
Listing 9.2: .github/workflows/deploy.yml — complete pipeline

```
name: CI/CD Pipeline

# Trigger on push to main branch
on:
  push:
    branches:
      - main
  # Allow manual trigger from the Actions tab
  workflow_dispatch:

# Environment variables available throughout the workflow
env:
  REGISTRY: ghcr.io
  IMAGE_NAME: ${{ github.repository }}

jobs:
  # -- Job 1: Build and Test --------------------------------
  build-and-push:
    name: Build & Push Docker Image
    runs-on: ubuntu-latest
```

```yaml

    permissions:
      contents: read
      packages: write

    outputs:
      image_tag: ${{ steps.meta.outputs.tags }}

    steps:
      # 1. Checkout repository code
      - name: Checkout code
        uses: actions/checkout@v4

      # 2. Set up Docker Buildx for multi-platform builds
      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v3

      # 3. Log in to GitHub Container Registry
      - name: Log in to GHCR
        uses: docker/login-action@v3
        with:
          registry: ${{ env.REGISTRY }}
          username: ${{ secrets.GHCR_USERNAME }}
          password: ${{ secrets.GHCR_PAT }}

      # 4. Extract metadata (tags and labels) for Docker
      - name: Extract Docker metadata
        id: meta
        uses: docker/metadata-action@v5
        with:
          images: ${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}
          tags: |
            type=ref,event=branch
            type=sha,prefix={{branch}}-
            type=raw,value=latest,enable={{is_default_branch}}

      # 5. Build and push Docker image
      - name: Build and push image
        uses: docker/build-push-action@v5
        with:
          context: .
          push: true
          tags: ${{ steps.meta.outputs.tags }}
          labels: ${{ steps.meta.outputs.labels }}
          cache-from: type=gha
          cache-to: type=gha,mode=max

  # -- Job 2: Deploy to OCI --------------------------------
  deploy:
    name: Deploy to OCI Instance
    runs-on: ubuntu-latest
```

```
    needs: build-and-push   # Only runs if build-and-push succeeded

    steps:
      # 1. Deploy via SSH
      - name: Deploy via SSH
        uses: appleboy/ssh-action@master
        with:
          host: ${{ secrets.OCI_HOST }}
          username: ${{ secrets.OCI_USER }}
          key: ${{ secrets.OCI_SSH_KEY }}
          port: 22
          script: |
            # Navigate to app directory
            cd ~/apps/my-app

            # Log in to GHCR
            echo "${{ secrets.GHCR_PAT }}" | \
              docker login ghcr.io -u "${{ secrets.GHCR_USERNAME }}"
                 --password-stdin

            # Pull the latest image
            docker compose pull

            # Restart containers (zero-downtime if using --no-deps)
            docker compose up -d --remove-orphans

            # Clean up old images to save disk space
            docker image prune -f

            # Verify containers are running
            docker compose ps

      # 2. Health check after deployment
      - name: Health check
        run: |
          sleep 15
          curl -f http://${{ secrets.OCI_HOST }}:80/health || exit 1
          echo "Deployment successful!"
```

9.3 Workflow Explained



9.4 Handling Environment Variables in Production

Your `.env` file on the OCI instance holds sensitive runtime values. Update the `docker-compose.yml` to read from it:

Listing 9.3: Using .env file in docker-compose.yml

```
services:
  app:
    env_file:
      - .env      # Reads variables from ~/apps/my-app/.env
```

Never Commit .env Files

Add `.env` to your `.gitignore`. Commit only `.env.example` with placeholder values to guide other developers.

Chapter 10

Your First Deployment

10.1 Pre-Deployment Checklist

Before pushing code, verify every item:

Done?	Item
☐	OCI instance is **Running** and SSH accessible
☐	Docker is installed and running on OCI instance
☐	`docker-compose.yml` is committed to the repository
☐	`Dockerfile` is committed to the repository
☐	All 7 GitHub secrets are configured
☐	`.env` file exists on OCI instance at `~/apps/my-app/.env`
☐	`.github/workflows/deploy.yml` is committed
☐	Security list allows ports 22, 80, 443

10.2 Triggering the First Build

Listing 10.1: Push code to trigger the pipeline

```
# Ensure all files are staged and committed
git add .
git commit -m "Add Docker and CI/CD configuration"
git push origin main
```

10.3 Monitoring the Pipeline

1. Navigate to your GitHub repository.
2. Click the **Actions** tab.

3. You should see **CI/CD Pipeline** running (yellow spinner).
4. Click on it to see real-time logs for each step.

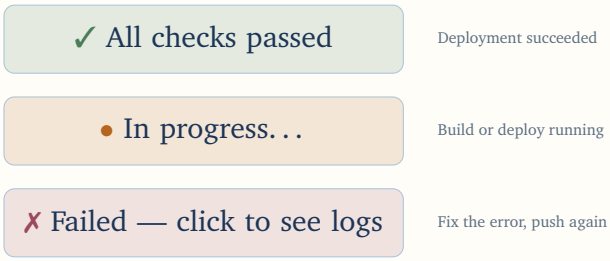


Figure 10.1: GitHub Actions run status indicators

10.4 Verifying the Deployment

Listing 10.2: Verify the deployment on the OCI instance

```
# SSH into your OCI instance
ssh ubuntu@<PUBLIC_IP>

# Check running containers
docker compose -f ~/apps/my-app/docker-compose.yml ps

# Expected output:
# NAME         IMAGE                              STATUS     PORTS
# my_app       ghcr.io/user/my-app:latest         Up 2m
    0.0.0.0:8000->8000/tcp
# my_db        postgres:15-alpine                 Up 2m      5432/tcp
# my_nginx     nginx:alpine                       Up 2m
    0.0.0.0:80->80/tcp

# Test the application
curl http://localhost:80/health
```

Chapter 11

Monitoring and Maintenance

11.1 Viewing Container Logs

Listing 11.1: Common Docker log commands

```
# View logs for all services
docker compose logs

# Follow logs in real time (like tail -f)
docker compose logs -f

# View logs for a specific service
docker compose logs -f app

# View last 100 lines
docker compose logs --tail=100 app
```

11.2 Container Resource Usage

The OCI Free tier gives you only **1 GB RAM**. Monitor usage:

Listing 11.2: Monitor resource usage

```
# Real-time resource stats for all containers
docker stats

# One-shot snapshot
docker stats --no-stream

# System-level memory check
free -h
df -h
```

Memory Limit — 1 GB RAM

With 1 GB RAM, run only what you need. PostgreSQL + a Python app + Nginx typically use 600–800 MB. Adding more services may cause OOM (Out-of-Memory)

kills. Use `mem_limit` in `docker-compose.yml` to cap each service.

Listing 11.3: Set memory limits in docker-compose.yml

```
services:
  app:
    mem_limit: 256m
    memswap_limit: 256m
  db:
    mem_limit: 256m
    memswap_limit: 256m
  nginx:
    mem_limit: 64m
```

11.3 Automatic Container Restarts

All services use `restart: unless-stopped`. This means:

- Containers restart automatically after a crash.
- Containers start on boot if Docker starts on boot (`systemctl enable docker`).
- Containers **do not** restart if you deliberately stop them (`docker compose stop`).

11.4 Managing Disk Space

Listing 11.4: Disk space management

```
# View Docker disk usage
docker system df

# Remove unused images, containers, networks (safe)
docker system prune -f

# Remove all unused images including tagged (caution!)
docker image prune -a -f

# Remove dangling volumes (caution: data loss)
docker volume prune -f
```

11.5 Updating the Application

Every `git push` to `main` triggers a new build and deploy automatically. For a manual deploy without a code change:

Listing 11.5: Manually trigger the workflow

```
# Via GitHub CLI
gh workflow run deploy.yml

# Or via the GitHub UI:
# Actions -> CI/CD Pipeline -> Run workflow -> Run workflow
```

Chapter 12

Troubleshooting

12.1 Troubleshooting Decision Tree

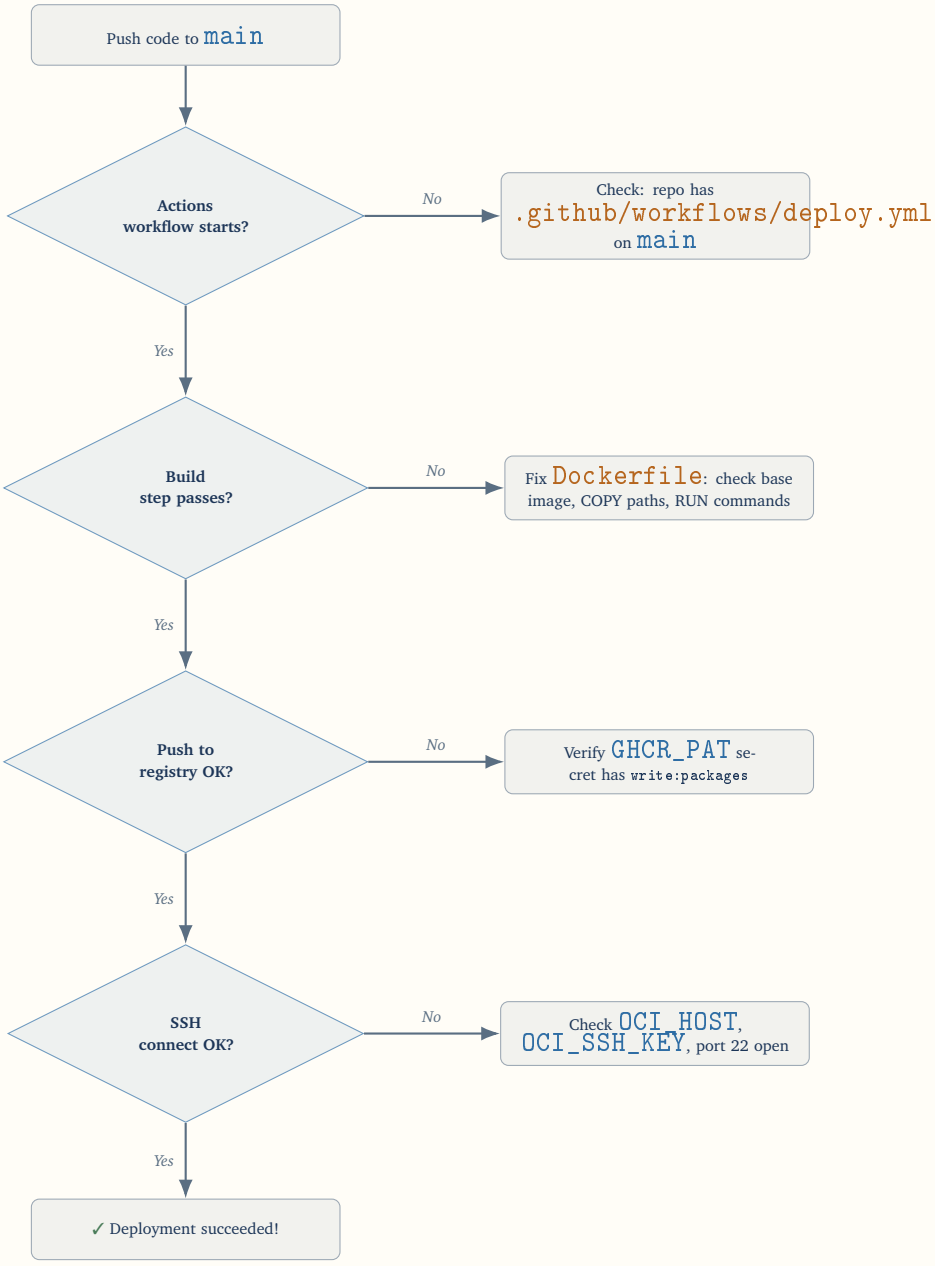


Figure 12.1: Deployment troubleshooting decision tree

12.2 Common Errors and Fixes

Error Message	Fix
permission denied (publickey)	Verify the private key in `OCI_SSH_KEY` secret matches the public key in `~/.ssh/authorized_keys` on OCI.
docker: Got permission denied	Run `sudo usermod -aG docker $USER` then log out and back in.
denied: installation not allowed	Your PAT does not have `write:packages` scope. Regenerate it.
No space left on device	Run `docker system prune -f` and check `df -h`.
OOMKilled (exit code 137)	Container exceeded memory. Add `mem_limit` or reduce services.
Connection refused port 80	OCI Security List missing ingress rule for port 80, or Nginx not running.
docker-compose: command not found	Use `docker compose` (space, no hyphen) — the plugin syntax.
image not found locally	The pull failed. Check registry credentials and image name in `docker-compose.yml`.

12.3 Checking Workflow Logs

1. Go to GitHub repository → **Actions** tab.
2. Click the failed workflow run.
3. Expand the failed job, then the failed step.
4. Read the red error output carefully — the cause is almost always visible.

Enable Debug Logging

Add these two secrets to enable verbose GitHub Actions logging:

```
ACTIONS_RUNNER_DEBUG   true
ACTIONS_STEP_DEBUG     true
```

Chapter 13

Security Best Practices

13.1 Secrets Management

- **Never hardcode** credentials in `Dockerfile`, `docker-compose.yml`, or workflow YAML.
- Use **GitHub Secrets** for all sensitive values.
- Rotate secrets **regularly** (every 90 days minimum).
- Use **fine-grained PATs** with minimal required scopes.
- Consider **GitHub Environments** with approval gates for production deployments.

13.2 SSH Hardening

Listing 13.1: Harden SSH configuration on OCI instance

```
sudo nano /etc/ssh/sshd_config

# Recommended settings:
# PermitRootLogin no
# PasswordAuthentication no
# PubkeyAuthentication yes
# MaxAuthTries 3
# ClientAliveInterval 300
# ClientAliveCountMax 2

# Apply changes
sudo systemctl restart sshd
```

13.3 Docker Security

Listing 13.2: Security-focused Dockerfile practices

```
# Use specific version tags (not latest in production)
FROM python:3.11.8-slim

```

```
# Run as non-root user
RUN adduser --disabled-password --no-create-home appuser
USER appuser

# Drop all capabilities
# (set in docker-compose.yml)
```

Listing 13.3: Security settings in docker-compose.yml

```
services:
  app:
    security_opt:
      - no-new-privileges:true
    cap_drop:
      - ALL
    cap_add:
      - NET_BIND_SERVICE      # Only if binding to port < 1024
    read_only: true
    tmpfs:
      - /tmp
```

13.4 Keeping the System Updated

Listing 13.4: Automated security updates

```
# Install unattended-upgrades for automatic security patches
sudo apt-get install -y unattended-upgrades
sudo dpkg-reconfigure -plow unattended-upgrades
# Choose "Yes" to enable automatic updates

# Configure only security updates (edit if needed)
cat /etc/apt/apt.conf.d/50unattended-upgrades
```

Chapter 14

Advanced Topics

14.1 Using Docker Build Cache Effectively

GitHub Actions caches Docker build layers between runs, dramatically reducing build times:

Listing 14.1: Optimised cache configuration in workflow

```
- name: Build and push image
  uses: docker/build-push-action@v5
  with:
    context: .
    push: true
    tags: ${{ steps.meta.outputs.tags }}
    cache-from: type=gha           # Pull cache from GitHub cache
    cache-to: type=gha,mode=max    # Push all layers to cache
```

First Build is Slow

The first pipeline run has no cache. Subsequent runs reuse cached layers and can be 3–10? faster, especially for dependency installation steps.

14.2 Multi-Environment Deployments

Use GitHub **Environments** to support staging and production:

Listing 14.2: Multi-environment workflow structure

```
jobs:
  deploy-staging:
    environment: staging
    runs-on: ubuntu-latest
    if: github.ref == 'refs/heads/develop'
    steps:
      - name: Deploy to staging
        # ... deploy to staging OCI instance

  deploy-production:
    environment: production    # Requires approval in GitHub UI
```

```
    runs-on: ubuntu-latest
    if: github.ref == 'refs/heads/main'
    needs: deploy-staging
    steps:
      - name: Deploy to production
        # ... deploy to production OCI instance
```

14.3 Rolling Back a Deployment

If a bad deployment reaches production:

Listing 14.3: Manual rollback on OCI instance

```
# On the OCI instance, list recent images
docker images ghcr.io/<username>/my-app

# Run a specific older version
cd ~/apps/my-app
IMAGE=ghcr.io/<username>/my-app:<old-sha-tag>

# Update the compose file temporarily
docker compose stop app
docker run -d --name my_app_rollback \
  --env-file .env \
  -p 8000:8000 \
  $IMAGE

# Or update docker-compose.yml and restart
docker compose up -d
```

GitHub Rollback

You can also re-run a previous successful workflow from the **Actions** tab by clicking the old run and selecting **Re-run jobs**.

14.4 Nginx as a Reverse Proxy

For production use, route traffic through Nginx:

Listing 14.4: nginx/nginx.conf — basic reverse proxy

```
events {
    worker_connections 1024;
}

http {
    upstream app_backend {
        server app:8000;
    }
```

```
    server {
        listen 80;
        server_name _;

        location /health {
            access_log off;
            return 200 "OK\n";
        }

        location / {
            proxy_pass         http://app_backend;
            proxy_set_header   Host               $host;
            proxy_set_header   X-Real-IP          $remote_addr;
            proxy_set_header   X-Forwarded-For
               $proxy_add_x_forwarded_for;
            proxy_set_header   X-Forwarded-Proto $scheme;
            proxy_read_timeout 300s;
            client_max_body_size 20M;
        }
    }
}
```

Chapter 15

Quick Reference

15.1 Essential Commands Cheat Sheet

Command	What It Does
`docker compose up -d`	Start all services in background
`docker compose down`	Stop and remove containers
`docker compose restart app`	Restart a specific service
`docker compose logs -f`	Stream live logs
`docker compose pull`	Pull latest images from registry
`docker compose ps`	List container status
`docker stats`	Real-time resource usage
`docker system prune -f`	Clean unused Docker resources
`docker exec -it my_app bash`	Open shell inside running container
`docker inspect my_app`	View full container configuration
`systemctl status docker`	Check Docker daemon status
`docker compose config`	Validate compose file syntax

15.2 GitHub Actions Workflow Triggers

Trigger	When It Fires
`push: branches: [main]`	Every push to `main`
`pull_request: branches: [main]`	Every PR targeting `main`
`schedule: cron: '0 2 * * *'`	Daily at 02:00 UTC
`workflow_dispatch:`	Manual trigger from Actions UI
`release: types: [published]`	When a GitHub release is published

Chapter A

Complete File Listing

A.1 .gitignore

Listing A.1: .gitignore

```
# Environment files
.env
.env.local
.env.*.local

# Python
__pycache__/
*.py[cod]
*.so
.venv/
venv/
dist/
build/
*.egg-info/

# Logs
*.log
logs/

# Docker
docker-compose.override.yml

# OS
.DS_Store
Thumbs.db

# IDE
.idea/
.vscode/
*.swp
```

A.2 .env.example

Listing A.2: .env.example — commit this, not .env

```
# Application
SECRET_KEY=change_me_in_production
DEBUG=false
PORT=8000

# Database
DATABASE_URL=postgresql://appuser:password@db:5432/appdb
POSTGRES_DB=appdb
POSTGRES_USER=appuser
POSTGRES_PASSWORD=change_me_in_production

# Registry
GITHUB_USERNAME=your_github_username
```

Chapter B

OCI Always Free Limits Reference

Resource	Always Free Allowance
AMD Compute	2 ? VM.Standard.E2.1.Micro (1 OCPU, 1 GB RAM each)
ARM Compute	4 OCPUs + 24 GB RAM total (Ampere A1)
Block Storage	200 GB total
Object Storage	20 GB
Outbound Data	10 TB/month
Load Balancer	1 ? 10 Mbps
VCN	2 VCNs with internet connectivity
Autonomous DB	2 ? 20 GB databases

Tip: Use the ARM Instance

The ARM (Ampere A1) Always Free allocation (4 OCPUs / 24 GB RAM) is **significantly** more powerful than the AMD micro instance. If you do not need x86 compatibility, consider switching to the ARM instance for much better performance at zero cost.

Chapter C

Glossary

CI/CD Continuous Integration / Continuous Deployment. Automates building, testing, and deploying code changes.

Docker Platform for developing and running applications in isolated containers.

Docker Compose Tool for defining and running multi-container Docker applications using a YAML file.

Dockerfile A text file containing instructions for building a Docker image.

Container A running instance of a Docker image. Lightweight, isolated, reproducible.

Image A read-only template used to create containers. Stored in a registry.

Registry A storage and distribution service for Docker images (e.g., GHCR, Docker Hub).

GitHub Actions GitHub's built-in CI/CD platform that runs workflows in response to events.

Workflow A YAML file in `.github/workflows/` that defines automated jobs and steps.

Secret An encrypted variable stored in GitHub, available to workflows but hidden from logs.

OCI Oracle Cloud Infrastructure — cloud provider offering Always Free compute instances.

VCN Virtual Cloud Network — OCI's virtual networking layer.

GHCR GitHub Container Registry — free image storage integrated with GitHub.

PAT Personal Access Token — used to authenticate with GitHub APIs and registries.

SSH Secure Shell — encrypted protocol for remote server access.

OOM Out-of-Memory — when a process is killed because the system ran out of RAM.

Summary & Next Steps

You have now completed the full setup of a production-grade CI/CD pipeline on an OCI Always Free instance. Here is a recap of what was achieved:

1. **Provisioned** an OCI AMD instance with Ubuntu 22.04.
2. **Secured** SSH access and opened required firewall ports.
3. **Installed** Docker Engine and Docker Compose.
4. **Structured** the application with a Dockerfile and `docker-compose.yml`.
5. **Configured** GitHub Secrets for secure credential storage.
6. **Written** a GitHub Actions workflow that automatically builds, pushes, and deploys.
7. **Tested** the pipeline end-to-end.

Suggested next steps:

- Add **automated tests** (unit, integration) as a pre-deploy job.
- Configure **SSL/TLS** using Let's Encrypt with Certbot or Nginx Proxy Manager.
- Set up **monitoring** with Grafana + Prometheus or UptimeRobot.
- Explore migrating to the **ARM (Ampere A1)** instance for more resources.
- Implement **database backups** using a scheduled GitHub Actions workflow or `pg_dump` cron job.
- Use **GitHub Environments** to add a manual approval gate before production deploys.

You now have a fully automated, zero-cost deployment pipeline.

Every `git push` to `main` automatically builds and ships your code.